지난 5개월 동안 우리 팀은 **수동으로 코드 라인을 전혀 작성하지 않고** 소프트웨어 제품의 내부 베타를 구축하고 출시하는 실험을 진행해 왔습니다.

이 제품에는 내부 일일 사용자와 외부 알파 테스터가 배정되어 있고 출시, 배포, 고장, 수리 작업이 진행됩니다. 다른 점은 애플리케이션 로직, 테스트, CI 구성, 문서화, 관측 가능성, 내부 툴링 등 모든 코드 라인이 Codex에 의해 작성되었다는 점입니다. 수작업으로 코드를 작성했을 때 걸렸을 시간의 약 1/10 정도의 시간으로 작성된 것으로 추정됩니다.

사람이 조정하고, 에이전트가 수행

엔지니어링 속도를 몇 배로 높이는 데 필요한 것을 구축하기 위해 의도적으로 이 제약을 선택했습니다. 결국 백만 개 라인에 달하는 코드를 출시하는 데 몇 주가 걸렸습니다. 이를 위해 우리는 소프트웨어 엔지니어링 팀의 주된 업무가 더 이상 코드 작성에서 벗어나 환경을 설계하고, 의도를 명시하며, Codex 에이전트가 안정적인 작업을 수행할 수 있도록 피드백 루프를 구축하는 것으로 바뀔 때 어떤 변화가 발생하는지 이해해야 했습니다.

이 글에서는 에이전트 팀과 함께 완전히 새로운 제품을 구축하면서 얻은 교훈, 즉 무엇이 문제이고 무엇이 보완되어야 하는지, 그리고 정말 부족한 자원인 사람의 시간과 주의를 극대화하는 방법에 대해 설명합니다.

빈 git 리포지터리로 시작함

빈 리포지터리에 대한 첫 번째 커밋은 2025년 8월 말에 이루어졌습니다.

초기 스캐폴드(리포지터리 구조, CI 구성, 서식 지정 규칙, 패키지 관리자 설정, 애플리케이션 프레임워크)는 소수의 기존 템플릿 세트에 따라 GPT-5를 사용하여 Codex CLI에서 생성되었습니다. 에이전트에게 리포지터리에서 작업하는 방법을 알려주는 초기 AGENTS.md 파일도 Codex가 직접 작성했습니다.

시스템을 뒷받침할 사람이 작성한 기존의 코드가 없었습니다. 처음부터 리포지터리는 에이전트에 의해 형성되었습니다.

5개월 후 리포지터리에는 애플리케이션 로직, 인프라, 툴링, 문서화, 내부 개발자 유틸리티 전반에 걸쳐 약 백만 개 라인의 코드가 포함되어 있습니다. 그 기간 동안 Codex를 이끄는 단 세 명의 엔지니어로 구성된 소규모 팀이 약 1,500개의 pull request를 열고 병합했습니다. 이는 엔지니어 1인당 하루 평균 3.5개의 PRs 처리량에 해당하며 놀랍게도 현재 7명의 엔지니어로 팀이 성장하면서 처리량이 *증가*했습니다. 중요한 점은, 단지 출력을 위한 출력이 아니었다는 것입니다. 이 제품은 매일 사용하는 내부 파워 유저를 포함해 수백 명의 사용자가 내부적으로 사용해 왔습니다.

개발 과정 전반에 걸쳐 사람은 코드에 직접 기여한 적이 없습니다. 이는 **수동으로 작성**

한 코드가 없다는 팀의 핵심 철학이 되었습니다.

엔지니어의 역할 재정의하기

사람이 직접 코딩을 하지 않기 때문에 **시스템, 스캐폴딩, 레버리지에 중점을 둔 다른 종류의 엔지니어링 작업이 필요했습니다**.

초기 진행은 예상보다 더뎠습니다. 이는 Codex의 역량이 부족해서가 아니라 환경이 제대로 갖춰지지 않았기 때문입니다. 에이전트가 상위 목표를 달성하기 위한 도구, 추상화, 내부 구조가 부족했습니다. 엔지니어링 팀의 주요 업무는 에이전트가 유용한 업무를 수행할 수 있도록 지원하는 것이 되었습니다.

실제로는 큰 목표를 더 작은 빌딩 블록(설계, 코드, 검토, 테스트 등)으로 세분화하고 에이전트가 이러한 블록을 구성하도록 유도한 다음 이를 사용하여 더 복잡한 작업을 해결하는 깊이 우선 작업을 의미했습니다. 실패했을 때 "더욱 분발"하는 것으로는 문제를 해결할 수 없었습니다. 진전을 이룰 수 있는 유일한 방법이 Codex가 일을 하도록 만드는 것이었기 때문에, 엔지니어들은 항상 작업에 직접 투입되어 "어떤 기능이 누락되어 있으며, 에이전트가 읽기 쉽고 실행 가능하게 만들려면 어떻게 해야 할까?"라는 고민을 했습니다.

엔지니어가 작업을 설명하고, 에이전트를 실행하고, pull request를 열도록 허용하는 등 사람은 거의 전적으로 프롬프트를 통해 시스템과 상호작용합니다. PR을 완료하기 위해, Codex에게 로컬에서 자체 변경 사항을 검토하고, 로컬 및 클라우드에서 특정 에이전트 검토를 추가로 요청하고, 사람이나 에이전트가 제공한 피드백에 응답하고, 모든 에이전트 검토자가 만족할 때까지 반복하도록 지시합니다(사실상 랄프 위검 루프(Ralph Wiggum Loop)(새 창에서 열기)임). Codex는 사람이 CLI에 복사하여 붙여넣지 않아도 표준 개발 도구(gh, 로컬 스크립트, 리포지터리 내장 스킬)를 직접 사용하여 컨텍스트를 수집합니다.

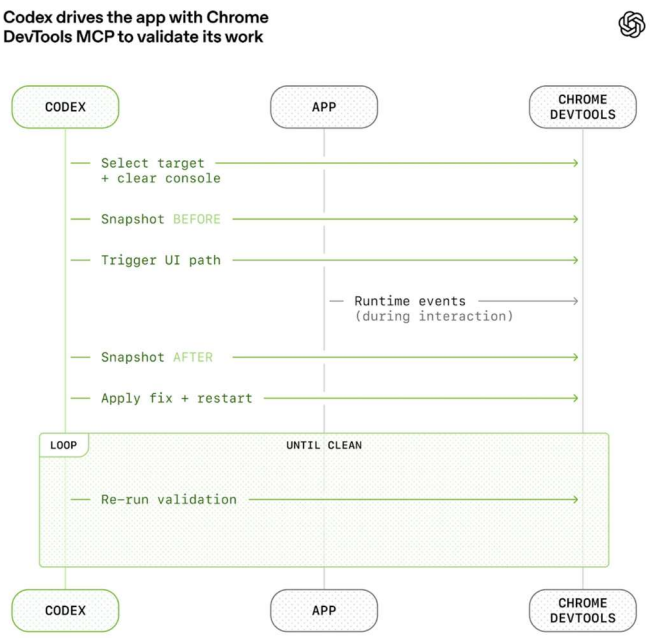
사람이 pull request를 검토할 수는 있지만, 반드시 필요한 작업은 아닙니다. 시간이 지남에 따라 거의 모든 검토 작업을 에이전트 간에 처리되도록 전환해 왔습니다.

애플리케이션의 가독성 향상

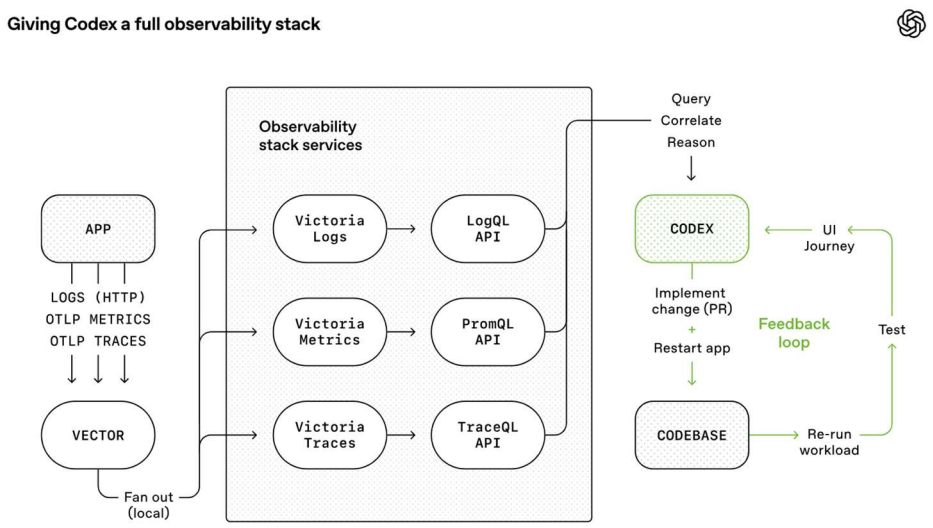
코드 처리량이 증가하면서, 인적 QA 역량에 병목 현상이 발생했습니다. 사람의 시간과 주의력이 고정된 제약이었기 때문에 애플리케이션 UI, 로그, 앱 메트릭 등을 Codex가 직접 읽고 이해할 수 있도록 하여 에이전트에 더 많은 기능을 추가하기 위해 노력해 왔습니다.

예를 들어, git worktree별로 앱을 부팅할 수 있게 하여 Codex가 변경사항마다 하나의 인스턴스를 실행하고 제어할 수 있도록 했습니다. 또한 Chrome DevTools Protocol을 에이전트 런타임에 연결하고 DOM 스냅샷, 스크린샷, 탐색 작업을 위한 스킬을 만들었습니

다. 이를 통해 Codex는 버그를 재현하고, 수정사항을 검증하며, UI 동작에 대해 직접 추론할 수 있었습니다.



관측 가능성 툴링에서도 마찬가지였습니다. 로그, 메트릭, 추적은 각 워크트리에 대해 일시적으로 유지되는 로컬 관측 가능성 스택을 통해 Codex에 노출됩니다. Codex는 해당 앱의 완전히 격리된 버전(로그와 메트릭 포함)에서 작업하며, 작업이 완료되면 로그와 메트릭은 삭제됩니다. 에이전트는 LogQL로 로그를 쿼리하고 PromQL로 메트릭을 쿼리할 수 있습니다. 이러한 컨텍스트가 주어지면 "서비스 시작이 800ms 이내에 완료"되도록 하거나 "이 네 가지 핵심 사용자 여정에서 어떤 스팬도 2초를 초과하지 않도록" 하는 등의 프롬프트를 다루기 쉬워집니다.



한 번의 Codex 실행으로 6시간 이상(종종 사람이 잠자는 동안) 한 가지 작업을 수행하는 경우가 많습니다.

리포지터리 지식을 기록 시스템으로 만듦

컨텍스트 관리는 에이전트가 크고 복잡한 작업을 효과적으로 수행하는 데 있어 가장 큰 과제 중 하나입니다. 초기에 우리가 배운 교훈 중 하나는 간단했습니다. **Codex에 1,000 페이지의 설명서가 아닌 맵을 제공**해야 한다는 것이었습니다.

"하나의 큰 AGENTS.md(새 창에서 열기)" 접근 방식을 시도해 보았지만 다음과 같이 예측 가능한 방식으로 실패했습니다.

- **컨텍스트는 희소 리소스.** 거대한 지침 파일로 인해 작업, 코드 및 관련 문서가 복잡해져서 에이전트가 주요 제약 조건을 놓치거나 잘못된 제약 조건에 맞게 최적화를 시작하게 됩니다.
- **지침이 너무 많으면 *지침이 되지 않음.*** 모든 것이 "중요"하다면 중요한 것은 아무것도 없습니다. 에이전트는 의도적으로 탐색하는 대신 로컬에서 패턴 매칭을 수행하게 됩니다.
- **순식간에 망가짐.** 획일적인 거대 매뉴얼은 낡은 규칙들의 무덤으로 변합니다. 에이전트는 무엇이 여전히 사실인지 알 수 없고, 사람들은 더 이상 이를 유지관리하지 않으며, 파일은 조용히 매력적인 골칫거리로 변합니다.
- **확인하기 어려움.** 단일 블롭은 기계적 점검(커버리지, 신선도, 소유권, 교차 연결)에 적합하지 않으므로 드리프트는 불가피합니다.

따라서 AGENTS.md를 백과사전으로 취급하는 대신 **목차**로 취급합니다.

리포지터리의 지식 베이스는 기록 시스템으로 취급되는 구조화된 docs/ 디렉터리에 있습니다. 짧은 AGENTS.md(약 100개 라인)는 컨텍스트에 삽입되어 주로 맵 역할을 하며, 다른 곳에 있는 더 심층적이고 신뢰할 수 있는 정보 소스를 알려줍니다.

일반 텍스트

```
1
AGENTS.md
2
ARCHITECTURE.md
3
docs/
4
├── design-docs/
5
│   ├── index.md
6
│   ├── core-beliefs.md
7
│   └── ...
8
├── exec-plans/
9
│   ├── active/
10
│   ├── completed/
11
```

```
|       └── tech-debt-tracker.md
12
├── generated/
13
|       └── db-schema.md
14
├── product-specs/
15
|       ├── index.md
16
|       ├── new-user-onboarding.md
17
|       └── ...
18
├── references/
19
|       ├── design-system-reference-llms.txt
20
|       ├── nixpacks-llms.txt
21
|       ├── uv-llms.txt
22
|       └── ...
23
├── DESIGN.md
```

24

├── FRONTEND.md

25

├── PLANS.md

26

├── PRODUCT_SENSE.md

27

├── QUALITY_SCORE.md

28

├── RELIABILITY.md

29

└── SECURITY.md

리포지터리 내 지식 저장소의 레이아웃.

설계 문서화는 검증 상태와 에이전트 우선 운영 원칙을 정의하는 몇 가지 핵심 신념을 포함하여 분류되고 색인화됩니다. 아키텍처 문서화(새 창에서 열기)는 도메인과 패키지 레이어링의 최상위 맵을 제공합니다. 품질 문서는 각 제품 도메인과 아키텍처 레이어에 등급을 매기며, 시간이 지남에 따라 격차를 추적합니다.

계획은 일급 아티팩트로 간주됩니다. 일시적이며 간단한 계획은 작은 변경에 사용되며, 복잡한 작업은 진행 상황과 의사결정 로그와 함께 실행 계획(새 창에서 열기)에 포함되어 리포지터리에 저장됩니다. 진행 중인 계획, 완료된 계획, 그리고 알려진 기술 부채는 모두 버전화되어 관리되고 동일한 위치에 배치되어 에이전트가 외부 상황에 의존하지 않고 작업할 수 있습니다.

이를 통해 **점진적으로 공개**할 수 있습니다. 에이전트는 처음부터 부담을 느끼지 않고 작고 안정적인 진입 지점에서 시작하여 다음 단계로 나아갈 수 있습니다.

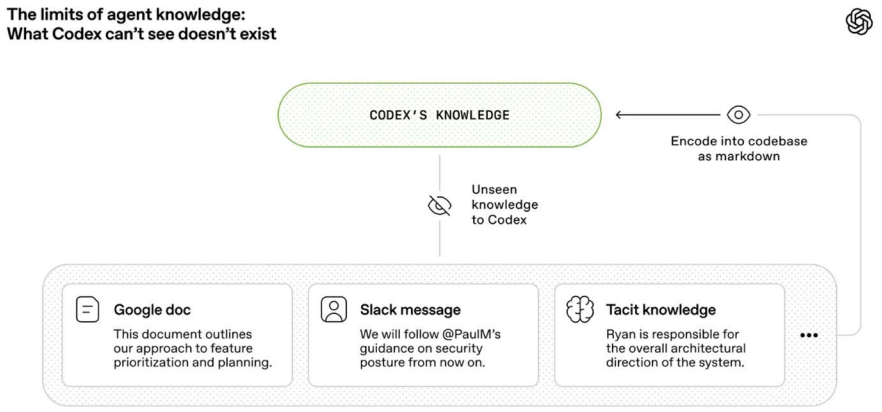
우리는 이를 기계적으로 시행합니다. 전용 린터와 CI 작업은 지식 베이스가 최신 상태이고, 교차 링크되어 있으며, 올바르게 구성되어 있는지 검증합니다. 반복 실행되는 "doc-gardening" 에이전트가 실제 코드 동작을 반영하지 않는 오래되거나 더 이상 유효하지 않은 문서화를 검토하여 수정용 pull request를 엽니다.

에이전트의 가독성이 목표

코드베이스가 발전함에 따라 Codex의 설계 의사결정을 위한 프레임워크도 진화할 필요가 있었습니다.

리포지터리가 전적으로 에이전트에 의해 생성되었기 때문에, 우선적으로 *Codex의 가독성*에 최적화되어 있습니다. 팀의 목표가 신입 엔지니어링 직원을 위해 코드의 탐색성을 개선하는 것과 마찬가지로, 인간 엔지니어의 목표는 에이전트가 **리포지터리 자체에서 직접** 전체 비즈니스 도메인에 대해 추론할 수 있도록 하는 것이었습니다.

에이전트의 관점에서 보면 실행 중에 컨텍스트 내에서 액세스할 수 없는 것은 사실상 존재하지 않는 것입니다. Google Docs, 채팅 스레드 또는 사람들의 머릿속에 있는 지식은 시스템에서 접근할 수 없습니다. 리포지터리 내의 버전 관리되는 아티팩트(예: 코드, 마크다운, 스키마, 실행 계획)에만 접근할 수 있습니다.



시간이 지나면서 점점 더 많은 컨텍스트를 리포지터리에 추가해야 한다는 것을 알게 되었습니다. Slack 토론에서 팀은 아키텍처 패턴에 대해 의견을 모았습니다. 에이전트가 검색할 수 없다면, 3개월 후에 입사한 신입 사원이 알 수 없는 것과 마찬가지로 판독이 불가능하다는 것입니다.

Codex에 더 많은 컨텍스트를 제공한다는 것은 임시 지침으로 너무 많은 부담을 주는 것이 아니라 올바른 정보를 정리하고 노출하여 에이전트가 추론할 수 있도록 한다는 의미입니다. 제품 원칙, 엔지니어링 규범, 팀 문화(이모지 선호도 포함)에 대해 새로운 팀원을 합류시키듯이 에이전트에게 이 정보를 제공하면 더 나은 결과물을 얻을 수 있습니다.

이 구상은 많은 절충점을 명확히 해주었습니다. 우리는 리포지터리 내에서 완전히 내부화하고 추론할 수 있는 종속성과 추상화를 선호했습니다. "지루하다"고 자주 묘사되는 기

술은 결합성, API 안정성, 학습 설정 내 표현 덕분에 에이전트가 모델링하기 더 쉬운 경향이 있습니다. 어떤 경우에는 공개 라이브러리의 불투명한 업스트림 동작에 맞춰 작업하는 것보다 에이전트가 기능의 일부 하위 집합을 다시 구현하는 것이 더 저렴했습니다. 예를 들어, 범용 p-limit 스타일 패키지를 가져오는 대신 자체적인 map-with-concurrency 헬퍼를 구현했습니다. 이는 OpenTelemetry 계측과 긴밀하게 통합되어 있으며, 테스트 커버리지가 100%이고, 런타임이 기대하는 방식대로 정확히 동작합니다.

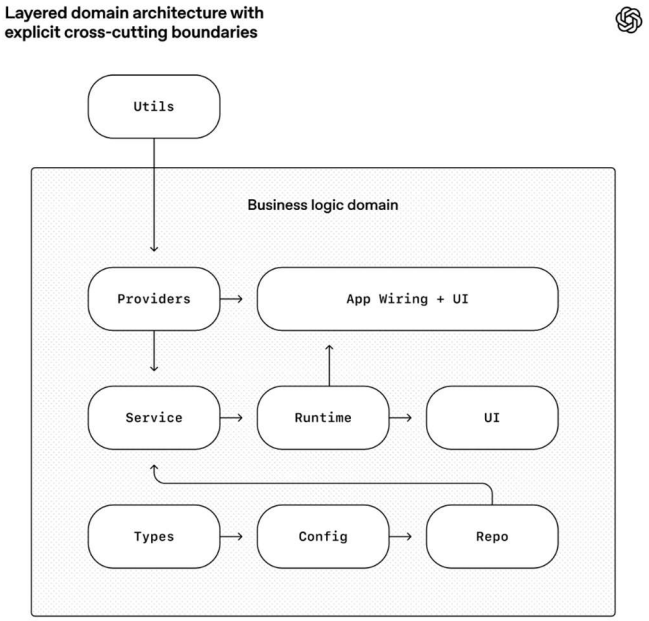
에이전트가 검사하고 검증하며 직접 수정할 수 있는 형태로 시스템을 더 많이 끌어오면 Codex뿐만 아니라 코드베이스에서 작업하는 다른 에이전트(예: Aardvark)에서도 레버리지를 높일 수 있습니다.

아키텍처 및 취향 강제 적용

문서화만으로는 에이전트가 생성한 코드베이스의 완전히 일관되게 유지할 수 없습니다. **구현을 세세하게 관리하지 않고 불변 조건을 강제 적용함으로써 기초를 튼튼히 유지하면서 에이전트가 빠르게 출시할 수 있도록 합니다.** 예를 들어, Codex가 경계에서 데이터 형태를 파싱(새 창에서 열기)하기를 원했지만 그 방법에 대해서는 구체적으로 명시하지 않았습니다(모델은 Zod를 선호하는 것 같지만, 특정 라이브러리를 지정하지 않았음).

에이전트는 엄격한 경계와 예측 가능한 구조(새 창에서 열기)를 갖춘 환경에서 가장 효과적이므로, 우리는 엄격한 아키텍처 모델을 중심으로 애플리케이션을 구축했습니다. 각 비즈니스 도메인은 엄격하게 검증된 종속성 방향과 제한된 허용 에지 집합을 사용하여 고정된 계층 집합으로 나뉩니다. 이러한 제약은 Codex가 생성한 맞춤형 린터와 구조적 테스트를 통해 기계적으로 적용됩니다.

아래 다이어그램은 다음 규칙을 보여줍니다. 각 비즈니스 도메인 내에서(예: 앱 설정) 코드는 고정된 레이어 집합(Types → Config → Repo → Service → Runtime → UI)을 통해서만 "전달"할 수 있습니다. 교차되는 문제(인증, 커넥터, 텔레메트, 기능 플래그)는 하나의 명시적인 인터페이스인 Providers를 통해 유입됩니다. 그 외의 모든 것은 허용되지 않으며 기계적으로 강제 적용됩니다.



이러한 종류의 아키텍처는 보통 수백 명의 엔지니어가 확보될 때까지 미루는 경우가 많습니다. 코딩 에이전트에는 초기 전제 조건이 있습니다. 제약 조건이 있어야 성능 저하나 아키텍처 드리프트 없이 속도를 낼 수 있습니다.

실제로 맞춤형 린터와 구조적 테스트, 그리고 소수의 '취향 불변성'을 통해 이러한 규칙을 적용합니다. 예를 들어, 맞춤형 린트를 사용하여 구조화된 로깅, 스키마 및 유형의 명명 규칙, 파일 크기 제한, 플랫폼별 안정성 요구 사항을 정적으로 적용합니다. 린트가 맞춤형이므로 오류 메시지를 작성하여 에이전트 컨텍스트에 수정 지침을 주입합니다.

사람 중심의 워크플로에서는 이러한 규칙들이 지나치게 세세하거나 제한적으로 느껴질 수 있습니다. 에이전트를 사용하면 효과를 몇 배로 늘릴 수 있습니다. 일단 인코딩되면 한 번에 어디에나 적용됩니다.

동시에 제약 조건이 중요한 부분과 그렇지 않은 부분을 명확히 구분합니다. 이는 대규모 엔지니어링 플랫폼 조직을 이끄는 것과 비슷합니다. 중앙에서 경계를 설정하고 로컬에서는 자율성을 허용하는 것입니다. 경계, 정확성, 재현성이 매우 중요할 경우 그러한 경계 내에서 팀이나 에이전트에게 솔루션을 표현하는 방식에 상당한 자유를 부여할 수 있습니다.

결과로 생성된 코드가 항상 인간의 문체 선호도와 일치하지 않을 수 있지만, 문제가 되지 않습니다. 출력이 정확하고, 유지관리 가능하며, 앞으로 에이전트 실행 시 읽기 쉽다면 기준을 충족하는 것입니다.

인간의 취향은 시스템에 지속적으로 피드백됩니다. 리뷰 코멘트, 리팩터링 pull request, 사용자 측 버그는 문서화 업데이트로 기록되거나 툴링에 직접 인코딩됩니다. 문서화가 부족한 경우 규칙을 코드로 승격합니다.

병합 철학을 변화시키는 처리량

Codex의 처리량이 증가함에 따라 기존의 많은 엔지니어링 규범이 오히려 역효과를 내기 시작했습니다.

리포지터리는 최소한의 차단 병합 게이트로 운영됩니다. pull request는 수명이 짧습니다. 테스트 불안정성은 진행을 무기한으로 막기보다는 후속 실행으로 해결하는 경우가 많습니다. 에이전트 처리량이 사람의 주의력을 훨씬 초과하는 시스템에서는 수정 비용이 저렴하고 대기 시간은 오래 걸립니다.

이는 처리량이 적은 환경에서는 적합하지 않습니다. 여기에서 종종 적절한 절충안이 필요합니다.

"에이전트 생성"의 실제 의미

코드베이스가 Codex 에이전트에 의해 생성된다는 것은 코드베이스에 있는 모든 것을 의미합니다.

에이전트가 생성하는 것:

- 제품 코드 및 테스트
- CI 구성 및 릴리스 툴링
- 내부 개발자 도구
- 문서화 및 설계 내역
- 평가 하네스
- 리뷰 코멘트 및 응답
- 리포지터리 자체를 관리하는 스크립트
- 생산 대시보드 정의 파일

사람은 항상 루프에 남아 있지만, 예전과는 다른 추상화 레이어에서 작업합니다. 업무를 우선시하고, 사용자 피드백을 수용 기준으로 바꾸며, 결과를 검증합니다. 에이전트가 어려움을 겪으면 이를 신호로 간주하여 도구, 가드레일, 문서화 등 무엇이 누락되었는지 파악하고 항상 Codex 자체에서 수정사항을 작성하여 리포지터리에 다시 제공합니다.

에이전트는 우리의 표준 개발 도구를 직접 사용합니다. 에이전트는 리뷰 피드백을 가져오고, 인라인으로 응답하고, 업데이트를 푸시하며, 종종 자체 pull request를 스쿼시하고 병합하기도 합니다.

자율성 수준의 증가

테스트, 검증, 리뷰, 피드백 처리, 복구 등 더 많은 개발 루프가 시스템에 직접 인코딩되면서 리포지터리는 최근 Codex가 새로운 기능을 처음부터 끝까지 주도할 수 있는 중요한 임계점을 넘어섰습니다.

이제 에이전트는 한 번의 프롬프트를 통해 다음을 수행할 수 있습니다.

- 코드베이스의 현재 상태 검증
- 보고된 버그 재현
- 실패 상황을 시연하는 동영상 녹화
- 수정사항 구현
- 애플리케이션을 실행하여 수정사항 검증
- 해결책을 보여 주는 두 번째 동영상 녹화
- pull request 열기
- 에이전트 및 사람 피드백에 응답하기
- 빌드 실패 감지 및 수정
- 판단이 필요한 경우에만 사람에게 에스컬레이션
- 변경사항 병합

이러한 동작은 이 리포지터리의 특정 구조와 툴링에 따라 크게 달라지며, 적어도 아직은 유사한 투자 없이 일반화할 수 있다고 가정해서는 안 됩니다.

엔트로피 및 가비지 컬렉션

에이전트의 완전한 자율성은 새로운 문제를 야기하기도 합니다. Codex는 리포지터리에 이미 존재하는 패턴, 심지어 불균일하거나 최적 상태가 아닌 패턴도 복제합니다. 따라서 시간이 지나면 필연적으로 드리프트가 발생합니다.

처음에는 사람들이 이를 수동으로 처리했습니다. 예전에는 우리 팀이 매주 금요일(일주일의 20%)마다 "AI 슬로프"를 정리하느라 시간을 보냈습니다. 예상대로 그 방식은 확장되지 않았습니다.

대신, "황금 원칙"이라는 것을 리포지터리에 직접 인코딩하고 정기적인 정리 프로세스를 구축하기 시작했습니다. 실행에서도 코드베이스의 가독성과 일관성을 유지하도록 설계된 규칙입니다. 예를 들어, (1) 불변 조건을 중앙에서 관리하기 위해 직접 만든 헬퍼보다 공유 유틸리티 패키지를 선호하며 (2) "YOLO-style"로 데이터를 탐색하지 않고, 경계를 검증하거나 유형 지정된 SDK에 의존하여 추측한 형태를 바탕으로 에이전트가 실수로 구축하지 못하도록 합니다. 정기적으로 편차를 검사하고, 품질 등급을 업데이트하며, 대상 리팩터링 pull request를 생성하는 Codex 백그라운드 작업을 운영합니다. 대부분은 1분 이내에 검토할 수 있으며 자동으로 병합됩니다.

이것은 가비지 컬렉션처럼 작동합니다. 기술 부채는 고금리 대출과 같아서, 이자가 쌓여 고통스럽게 한꺼번에 갚는 것보다 조금씩 꾸준히 갚아나가는 편이 훨씬 낫습니다. 사람의 취향은 한 번 포착되면 코드의 모든 라인에 지속적으로 반영됩니다. 이렇게 하면 나쁜 패턴이 코드베이스에 며칠 또는 몇 주 동안 퍼지지 않도록 매일 포착하고 해결할 수 있습니다.

우리가 여전히 배우고 있는 점

이 전략은 지금까지 OpenAI에서 내부 출시와 도입 단계까지 잘 적응되어 왔습니다. 실제 사용자를 위한 실제 제품을 구축함으로써 투자를 현실에 안착시키고 장기적인 유지관리 가능성을 확보하는 데 도움이 되었습니다.

우리가 아직 알지 못하는 것은 완전한 에이전트 생성 시스템에서 아키텍처 일관성이 수년에 걸쳐 어떻게 진화하는지에 대한 것입니다. 우리는 사람의 판단이 가장 큰 영향력을 발휘하는 부분과 그 판단을 인코딩하여 복합적으로 활용하는 방법을 여전히 배우고 있습니다. 또한 시간이 지나면서 모델의 기능이 계속 향상됨에 따라 이 시스템이 어떻게 발전할지 알 수 없습니다.

분명해진 것은 소프트웨어를 구축하는 데는 여전히 규율이 필요하지만, 규율은 코드보다는 스캐폴딩에서 더 많이 드러난다는 점입니다. 코드베이스의 일관성을 유지하는 툴링, 추상화, 피드백 루프는 점점 더 중요해지고 있습니다.

현재 가장 어려운 과제는 에이전트가 우리의 목표인 복잡하고 안정적인 소프트웨어를 대규모로 구축하고 유지관리하는 데 도움이 되는 환경, 피드백 루프, 제어 시스템을 설계하는 것입니다.

Codex와 같은 에이전트가 소프트웨어 라이프사이클에서 더 많은 부분을 차지함에 따라 이러한 질문은 더욱 중요해질 것입니다. 몇 가지 초기 교훈을 공유하여 직접 구축할 수 있도록 어디에 노력을 투입해야 할지 판단하는 데 도움이 되길 바랍니다.